

实验二ECDSA签名验证与secp256k1密码学库的Bug修复与性能优化研究报告

一、引言

比特币核心使用的secp256k1椭圆曲线密码库是整个比特币生态系统最基础的密码学组件。该库实现了ECDSA（椭圆曲线数字签名算法）的签名、验证以及相关的椭圆曲线运算。ECDSA验证的核心数学公式为 $R' = s^{-1}(eG + rP)$ ，验证者计算该点后，检查其x坐标 r' 是否等于签名中的 r 值。验证成立的条件是：

$s^{-1}(eG + rP) = kG = R$ ，其中 $e = \text{hash}(m)$ 是消息哈希， $P = dG$ 是公钥， (r, s) 是签名。这个公式背后，隐藏着复杂的实现细节：有限域运算的数值边界管理、椭圆曲线标量乘法的性能优化、跨平台数据类型的可移植性保障等。任何一处的疏忽都可能导致安全漏洞或性能瓶颈。

本报告基于对secp256k1仓库Pull Requests的深入分析，选取三个典型案例，分别从Bug修复和性能优化两个维度，揭示密码学库开发中“数学正确性”与“工程实现”之间的紧密联系。

二、Bug修复案例分析

2.1 案例一：fe_equal函数的magnitude边界错误（PR #1877）

问题描述

在secp256k1的有限域实现中，每个域元素（secp256k1_fe）都有一个magnitude字段，表示其数值的膨胀倍数。这是为了在多次乘法运算后延迟归一化以提升性能而设计的优化手段。

secp256k1_fe_equal(a, b)函数用于判断两个域元素是否相等。该函数的文档和前置检查（SECP256K1_FE_VERIFY_MAGNITUDE）声明参数b的magnitude上限为31。然而，实际实现中存在一个隐蔽的边界错误。

数学分析

fe_equal的简化实现如下：

代码块

```
1  secp256k1_fe_negate(&na, a, 1);    // na = -a, magnitude变为2
2  secp256k1_fe_add(&na, b);          // na = na + b
```

这里有两个关键的数学约束：

`secp256k1_fe_negate(&na, a, 1)`将 a ($\text{magnitude} \leq 1$) 取反，结果 na 的 magnitude 变为 $m+1=2$ 。

`secp256k1_fe_add(&na, b)`要求两个操作数的 magnitude 之和不超过32（即 $r \rightarrow \text{magnitude} + a - > \text{magnitude} \leq 32$ ）。

因此，当 b 的 magnitude 为31时：

na 的 $\text{magnitude} = 2$

总和 $= 2 + 31 = 33 > 32$

这违反了`fe_add`的内部安全约束，会导致VERIFY构建中的断言失败。正确的 b 的 magnitude 上限应为30，而非31。

用数学语言表达：

`fe_equal(a, b)` 实际执行： $-a + b$, $\text{magnitude}(-a) = \text{magnitude}(a) + 1 \leq 2$, 要求： $\text{magnitude}(-a) + \text{magnitude}(b) \leq 32$, $\therefore \text{magnitude}(b) \leq 30$

安全影响

虽然实际代码中所有传递给`fe_equal`的 b 参数的 magnitude 最大值仅为7，远低于30，因此该bug在当前代码中并不会触发实际错误。但这一错误反映了密码学库开发中的一个重要原则：文档声明与实现必须一致。如果未来有开发者依赖文档中“31”的声明来编写新代码，就可能引入潜在的安全漏洞。

该bug自2022年引入（commit 7d7d43c6dd），早于`fe_add`的 magnitude 断言被添加的时间。修复方案是将文档和检查中的上限从31调整为30。

与ECDSA的关联

在ECDSA验证中，`fe_equal`可能被用于比较椭圆曲线点的坐标。虽然当前调用未触发此边界错误，但这一案例说明：即使是简单的相等比较，在有限域算术的上下文中也涉及复杂的 magnitude 管理。

2.2 案例二：WNAF算法中int类型大小的错误假设（PR #1772）

问题描述

`secp256k1_ecmult_wnaf`函数（WNAF, Windowed Non-Adjacent Form）是椭圆曲线标量乘法的核心算法。该函数包含一个断言 `VERIFY_CHECK(w <= 31)`，隐含假设了`int`类型至少有32个值位（value bits）。

然而，C语言标准只保证`int`至少有16个值位。在16位平台上，当`ECMULT_WINDOW_SIZE > 16`时，代码将无法正确运行。

数学分析

WNAF是一种将标量 k 表示为带符号二进制串的方法：
$$k = \sum_{i=0}^{\text{len}} \text{wnaf}_i \cdot 2^i$$

其中每个非零系数 wnaf_i 是奇数，且满足 $|\text{wnaf}_i| < 2^w$ ，在secp256k1中，窗口大小 w 可配置范围为2到24（ECMULT_WINDOW_SIZE）。当 $w=24$ 时，系数的取值范围为：

$$-(2^{24} - 1) \leq \text{wnaf}_i \leq 2^{24} - 1,$$

这需要至少25位（24个值位 + 1个符号位）来完整表示。如果int只有16位，数值范围仅为 $[-32768, 32767]$ ，远不足以存储 $2^{24} - 1 = 16,777,215$ 这样的值。

修复方案

将所有存储WNAF系数的变量从int改为int32_t：

代码块

```
1 // 修改前
2 int wnaf[256];
3
4 // 修改后
5 int32_t wnaf[256];
```

这确保了在任何平台上，WNAF系数都能被正确表示，不会发生溢出。位置相关的变量（如bit、len等）则保持为int类型。

与ECDSA的关联

WNAF算法直接影响ECDSA签名生成和验证中的核心运算—，即标量乘法 kG 和 rP 的计算效率与正确性。如果WNAF系数因整数溢出而计算错误，将直接导致签名验证失败或签名不可用。这一修复保证了secp256k1库在所有C语言平台上的行为一致性。

3.1 案例：批量点乘的算法自动选择与内存管理优化（PR #1789）

问题背景

ECDSA验证的核心计算是：
$$R' = s^{-1} (eG + rP)$$

在比特币区块验证中，需要同时验证数千笔交易的签名。这相当于要计算多个形如

$$\sum \text{scalar}_i \cdot \text{point}_i$$

的表达式。传统做法是逐个验证每个签名，效率低下。

PR #1789对ecmult_multi进行了两项重大改进：

用内部malloc替换scratch space，简化内存管理。

引入ABCD成本模型，动态选择最优算法。

数学分析：三种算法的复杂度对比

(1) Trivial算法

逐个计算每个标量乘法并累加：

时间复杂度： $O(n \cdot M)$ ，其中 M 是单次标量乘法的成本

空间复杂度： $O(1)$

适用于极少量点（ n 很小）或内存极度受限的场景

2) Strauss算法

利用窗口法和预计算表，将多个标量乘法合并处理：

时间复杂度： $O(n^2/\log n)$

空间复杂度： $O(n)$

适用于中等规模的点集（约5~150个点）

(3) Pippenger算法

将标量按比特位拆分，逐位处理所有点：

时间复杂度： $O(n/\log n)$

空间复杂度： $O(2w)$ ，其中 w 是窗口大小

适用于大规模的点集（约150个点以上）

ABCD成本模型

PR #1789引入的ABCD模型为每个算法建立了线性成本函数：

$$t(x) = C \cdot x + D$$

其中：

x 是批处理大小（点数）

C 是每点的边际成本（斜率）

D 是固定开销（截距）

算法的内存模型为：

$$m(x) = A \cdot x + B$$

其中 A 是每点内存占用， B 是固定内存开销。

算法选择逻辑：

在所有满足 $m(x) \leq \text{mem_limit}$ 的算法中

选择单位点耗时 $C+D/x$ 最小的那个

从校准数据可以看到各算法的ABCD参数：

算法	A (每点内存)	B (固定内存)	C (边际成本)	D (固定开销)
STRAUSS	~150	0	113	143
PIPPINGER _4	~50	~1158	100	1158
PIPPINGER _8	~43	~8775	69	8775
PIPPINGER _12	~43	~48783	80	48783

可以看到：

Pippenger算法的边际成本C更低（69 vs 113），适合大批量

但Pippenger的固定开销D更大（8775 vs 143），小批量时Strauss更优

随着窗口 w 增大，固定开销急剧增加，但边际成本下降

这就是为什么需要自动选择算法——没有一种算法在所有规模下都是最优的。

批处理机制

当内存限制无法容纳所有点时，secp256k1_ecmult_multi会自动将点集分成多个批次：

代码块

```
1  /* 伪代码 */
2  batch_size = secp256k1_ecmult_multi_batch_size(mem_limit);
3  for (offset = 0; offset < n_points; offset += batch_size) {
4      actual = min(batch_size, n_points - offset);
5      使用最优算法计算当前批次；
6      累加结果到 r；
7  }
```

与ECDSA验证公式的关联

批量验证多个签名的核心计算为：

$$\sum_{i=1}^n s_i^{-1} (e_i G + r_i P_i) = \left(\sum_{i=1}^n s_i^{-1} e_i \right) G + \sum_{i=1}^n (s_i^{-1} r_i) P_i$$

这正是 ecmult_multi 所计算的表达式——其中

$$\text{scalar_g} = \sum s_i^{-1} \cdot e_i, \quad \text{scalars}[i] = s_i^{-1} \cdot r_i, \quad \text{points}[i] = P_i$$

因此，PR #1789的性能优化直接提升了比特币区块验证的吞吐量。在大规模验证场景下，Pippenger 算法可将验证时间减少 30%~50%。

四、总结

通过对secp256k1库三个典型案例的分析，可以得出以下结论：

PR	类型	核心问题	数学原理	安全/性能影响
#1877	Bug修复	fe_equal的 magnitude边界应为30而非31	有限域运算的 magnitude管理：fe_negate使 magnitude+1，加法要求总和≤32	防止文档与实现不一致导致的潜在漏洞
#1772	Bug修复	WNAF系数依赖int为32位	WNAF系数范围 $[-(2w-1), 2w-1]$ $[-(2w-1), 2w-1]$ ，w最大24	保证跨平台可移植性
#1789	性能优化	批量点乘算法单一，缺乏自动选择	Strauss $O(n^2/\log n)$ vs Pippenger $O(n/\log n)$ + ABCD成本模型	批量验证速度提升 30~50%

这三个案例共同揭示了密码学库开发的核心原则：

数学正确性是安全的基础——#1877中magnitude边界的“1”之差，反映了有限域算术中细微但关键的数学约束；#1772中整数类型的假设，关乎WNAF算法数学定义的完整实现。

性能优化必须在不破坏数学正确性的前提下进行，#1789中引入的ABCD模型，本质上是在保证计算结果完全相同的前提下，通过算法复杂度分析来提升效率。

密码学库的工程实现与纯数学理论之间存在巨大鸿沟——从ECDSA的简洁公式 $s^{-1}(eG + rP)$ 到生产级代码，需要处理数据类型、内存管理、跨平台兼容性、性能优化等无数工程细节。任何一处的疏忽，都可能将数学上正确的算法变成工程上有漏洞的实现。